Optimization Behavior of Riemannian Optimizers for Hyperbolic WordNet Subtree Classification

Shengze Jiang, Xinran Li, Tianyi Zhu
CS-439 Optimization for machine learning mini-project

***Abstract*—Hyperbolic geometry is well suited for hierarchical data, but classifiers defined in the Poincar'e ball require optimization methods that respect manifold-constrained parameters. We study binary WORDNET subtree classification using pretrained Poincar'e embeddings and compare eight method–optimizer combinations: Euclidean and tangent-space logistic regression with SGD/Adam, and hyperbolic MLR (HMLR) with projected SGD, projected Adam, RSGD, or RAdam. Across four subtrees, four dimensions, and three seeds, HMLR variants win 15 out of 16 subtree–dimension cells. HMLR with RAdam achieves the best mean F_1 of 0.90, outperforming the strongest Euclidean baseline at 0.55. Riemannian optimizers also show smoother convergence and hyperplane trajectories on average, although projected methods remain competitive in some individual settings. These results support combining hyperbolic classifiers with geometry-aware optimization for hierarchical prediction.**

I. INTRODUCTION

Hierarchical data such as taxonomies and lexical hierarchies are often better represented in hyperbolic space than in Euclidean space, since the exponential volume growth of the Poincaré ball matches tree-like structure [1], [2]. Classifiers built directly in the Poincaré ball therefore provide a natural model for hierarchical prediction tasks, but their parameters must remain inside the ball, turning training into a constrained optimization problem on a Riemannian manifold.

We study this problem on binary WORDNET subtree classification using pretrained Poincaré embeddings. We compare Euclidean, tangent-space, and hyperbolic classifiers together with their corresponding optimizers. Our results show that HMLR is the strongest classifier family, and that RAdam gives the best average performance among HMLR optimizers.

II. LITERATURE REVIEW

A. Hyperbolic Representation Learning

Hierarchical structures such as taxonomies are difficult to represent with low distortion in low-dimensional Euclidean space [1]. Hyperbolic space is well suited for such data because its volume grows exponentially with radius, matching the branching structure of trees [3], [4]. Poincaré embeddings exploit this geometry and outperform Euclidean embeddings on reconstruction and link prediction over the WORDNET noun hierarchy [1], [5]. Ganea et al. further introduced hyperbolic neural layers in the Poincaré ball, including hyperbolic multinomial logistic regression (HMLR), which represents each class by a Poincaré hyperplane and scores inputs by signed hyperbolic distance [2]. Since HMLR improves over Euclidean and tangent-space MLR on WORDNET subtree classification, we use the same task and classifier family but study the optimization methods used to train it.

B. Riemannian Optimization

Training HMLR is a constrained optimization problem because its hyperplane points must remain inside the Poincaré ball. Projected-Euclidean methods apply ordinary SGD or Adam updates in ambient coordinates and project invalid iterates back into the ball [6], [1]. Riemannian methods instead use the local metric to convert Euclidean gradients into Riemannian gradients and update parameters with manifold operations such as retractions or exponential maps [7]. RAdam further extends Adam-style adaptive updates to manifolds by transporting momentum across tangent spaces [8]. Prior work notes possible instability of projected variants for hyperbolic models [2], but optimizer choice has not been isolated in a controlled WORDNET subtree experiment. We therefore compare projected SGD/Adam with RSGD/RAdam in terms of F_1, convergence, projection behavior, and hyperplane trajectories.

III. METHODS

We compare three ways of using pretrained Poincaré embeddings for WordNet subtree classification: direct Euclidean logistic regression, tangent-space logistic regression, and hyperbolic multinomial logistic regression (HMLR). This lets us study how classifier geometry and optimizer choice jointly affect performance and training behavior.

A. Poincaré Ball Geometry

The n-dimensional Poincaré ball is

$$\mathbb{D}^n = \{x \in \mathbb{R}^n : \|x\| < 1\},$$

equipped with the conformal Riemannian metric

$$g_x = \lambda_x^2 g_E, \qquad \lambda_x = \frac{2}{1 - \|x\|^2}.$$

As points approach the boundary, distances are stretched, giving hyperbolic space exponential volume growth and making it suitable for tree-like data [3], [1]. The induced distance is

$$d(x, y) = \cosh^{-1}\left(1 + 2\frac{\|x - y\|^2}{(1 - \|x\|^2)(1 - \|y\|^2)}\right). \quad (1)$$

We also use Möbius addition,

$$x \oplus y = \frac{(1 + 2\langle x, y\rangle + \|y\|^2)x + (1 - \|x\|^2)y}{1 + 2\langle x, y\rangle + \|x\|^2\|y\|^2}, \quad (2)$$

so that $-p_k \oplus x$ acts as the hyperbolic analogue of subtracting a hyperplane offset.

B. Euclidean and Tangent-Space Baselines

We include two Euclidean baselines to separate the effect of the representation from the effect of the classifier geometry. The direct Euclidean baseline treats each Poincaré embedding x_i simply as a vector in $\mathbb{R}^n$ and trains logistic regression with score

$$s(x_i) = w^\top x_i + b.$$

This ignores the non-Euclidean metric of the Poincaré ball, but tests whether the raw coordinates are already linearly separable.

The tangent-space baseline maps each embedding to the tangent space at the origin, which is a Euclidean vector space locally approximating the manifold around 0. For curvature $c = 1$, the logarithmic map at the origin is

$$z_i = \log_0(x_i) = \frac{2\tanh^{-1}(\|x_i\|)}{\|x_i\|} x_i \in T_0\mathbb{D}^n, \tag{3}$$

with the convention $\log_0(0) = 0$. Logistic regression is then applied in this tangent space:

$$s(z_i) = w^\top z_i + b.$$

Thus, this baseline uses hyperbolic geometry through the log-map representation, but the classifier and optimization problem remain Euclidean. It provides an intermediate comparison between direct Euclidean LR and the fully hyperbolic HMLR classifier. Both baselines are trained with SGD and Adam.

C. Hyperbolic MLR

For the fully hyperbolic classifier, we use the two-class HMLR of [2]. Each class $k \in \{0, 1\}$ is represented by a Poincaré hyperplane parameterized by a point $p_k \in \mathbb{D}^n$ and a normal vector $a_k \in T_{p_k}\mathbb{D}^n$. The class score is the signed hyperbolic distance to this hyperplane:

$$s_k(x) = \frac{\lambda_{p_k}\|a_k\|}{\sqrt{c}} \sinh^{-1}\left(\frac{2\sqrt{c}\langle -p_k \oplus x, a_k\rangle}{(1 - c\| - p_k \oplus x\|^2)\|a_k\|}\right), \tag{4}$$

with curvature $c = 1$ and $\lambda_{p_k} = 2/(1 - \|p_k\|^2)$. Class probabilities are obtained by applying a softmax over $s_0(x)$ and $s_1(x)$.

Since a_k belongs to the tangent space at the moving point p_k, we follow [2] and store a fixed vector $a_k^{(0)} \in T_0\mathbb{D}^n$, using

$$a_k = \frac{\lambda_0}{\lambda_{p_k}} a_k^{(0)}, \qquad \lambda_0 = 2.$$

Thus, $a_k^{(0)}$ is an unconstrained Euclidean parameter, while p_k is a manifold-valued parameter that must remain inside the Poincaré ball.

D. Optimization

All models minimize binary cross-entropy on pretrained embeddings $\{x_i\}_{i=1}^N$ and subtree labels $\{y_i\}_{i=1}^N$. The Euclidean and tangent-space baselines solve unconstrained optimization problems over Euclidean parameters $\theta = (w, b)$. HMLR instead has manifold-valued hyperplane points p_k and solves

$$\begin{aligned} \min_{\{p_k\},\{a_k^{(0)}\}} \quad & \frac{1}{N}\sum_{i=1}^{N} \ell(y_i, p(y \mid x_i)) \\ \text{s.t.} \quad & p_k \in \mathbb{D}^n, \quad k \in \{0, 1\}. \end{aligned} \tag{5}$$

This constraint is the key optimization difference between HMLR and standard logistic regression.

For HMLR, we compare projected SGD/Adam with RSGD/RAdam. Projected optimizers first update p_k as if it were an ordinary Euclidean vector, then project it back into $\mathbb{D}^n$ if the update leaves the ball. Riemannian optimizers instead use the geometry of the Poincaré ball during the update. Under the conformal metric $g_p = \lambda_p^2 g_E$, the Riemannian gradient is obtained by rescaling the Euclidean gradient,

$$\operatorname{grad}^R f(p) = \frac{1}{\lambda_p^2}\nabla^E f(p),$$

and the parameter is then moved on the manifold using a retraction or exponential-map step [7], [8]. Thus, projected methods correct invalid Euclidean steps after they occur, whereas Riemannian methods make geometry-aware updates from the start. Overall, our experiment compares direct Euclidean classification, tangent-space Euclidean classification, and fully hyperbolic classification with projected or Riemannian updates.

IV. Experiment

A. Experimental Setup

We study binary WordNet subtree classification over pretrained Poincaré embeddings. For each root synset r, nodes in its subtree are labelled positive and all other noun synsets are labelled negative. We use the same four roots as [2]: animal.n.01, mammal.n.01, group.n.01, and worker.n.01. Since the positive class is small, we use F_1 as the primary metric and split positive and negative examples separately into $80/20$ train/test sets.

We train separate Poincaré embeddings for $D \in 2, 3, 5, 10$ using the implementation of [1] and the best reconstruction checkpoint. We compare eight method–optimizer settings: Euclidean LR and log-map LR with SGD/Adam, and HMLR with projected SGD, projected Adam, RSGD, and RAdam. Euclidean LR treats Poincaré coordinates as ordinary features, log-map LR applies logistic regression after mapping points to $T_0\mathbb{D}^n$, and HMLR uses Poincaré hyperplanes with manifold-valued points p_k. Projected HMLR optimizers update these points in ambient coordinates and project back if needed, while RSGD/RAdam use Geoopt Riemannian updates [9], [7], [8]. All settings use learning rate 0.01, no weight decay, batch size 256, 30 epochs, and three seeds $0, 1, 2$.

B. Overall Results

Table I reports the aggregate comparison across all subtrees, dimensions, and seeds. HMLR with RAdam achieves the highest mean F_1 of 0.90, followed by HMLR with projected

Adam, RSGD, and projected SGD. The strongest Euclidean baseline is log-map LR with Adam, reaching $F_1 = 0.55$, while direct Euclidean LR with Adam reaches $F_1 = 0.48$. Thus, classifiers operating directly in the Poincaré ball generally exploit the pretrained hyperbolic embeddings more effectively than Euclidean or tangent-space classifiers. To avoid cherry-picking, Appendix A reports the complete seed-averaged F_1 matrices for all eight settings, four dimensions, and four subtrees.

TABLE I: Overall performance across four subtrees, four embedding dimensions, and three random seeds. Mean F_1 is averaged over all runs. "Wins" counts the number of subtree–dimension cells in which a setting obtains the highest mean F_1 after averaging over three seeds.

Setting	Mean F_1	Wins
H-MLR + RAdam	**0.90**	7
H-MLR + Proj. Adam	0.70	4
H-MLR + RSGD	0.68	1
H-MLR + Proj. SGD	0.56	3
Log-map LR + Adam	0.55	0
Euc-LR + Adam	0.48	1
Log-map LR + SGD	0.26	0
Euc-LR + SGD	0.16	0

C. Low-Dimensional Behavior

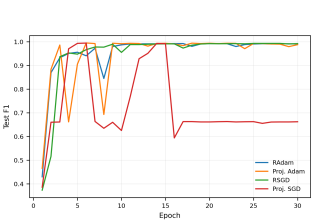
Embedding dimension strongly affects all methods: averaged over settings, mean F_1 increases from 0.254 at $D = 2$ to 0.772 at $D = 10$; the full dimension-wise averages are reported in Appendix A. However, the lowest-dimensional setting also highlights the benefit of hyperbolic classification. As shown in the complete subtree results in Appendix A, at $D = 2$ all four subtree tasks are won by HMLR variants: RAdam on ANIMAL.N.01, MAMMAL.N.01, and WORKER.N.01, and projected Adam on GROUP.N.01. This suggests that Poincaré hyperplanes can exploit the radial and angular structure of low-dimensional hyperbolic embeddings more effectively than Euclidean decision boundaries.

D. Optimization Behavior of HMLR

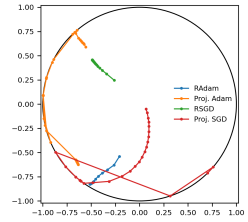
Within HMLR, Adam-based methods outperform SGD-based methods on average, as shown in Table I. RAdam reaches mean $F_1 = 0.90$, compared with 0.70 for projected Adam, while RSGD reaches 0.68, compared with 0.56 for projected SGD. Hence, Riemannian variants outperform their projected counterparts on average for both optimizer bases. However, the complete subtree–dimension results in Appendix A show that projected methods still win some individual cells. This suggests that the advantage of Riemannian optimization is an aggregate trend rather than uniform dominance in every setting.

E. Convergence and Stability

Figure 1 illustrates the convergence and trajectory behavior of the four HMLR optimizers. In Figure 1a, the Riemannian optimizers show smoother convergence, while projected Adam oscillates during early epochs and projected SGD eventually falls to a lower F_1. In Figure 1b, the trajectory of the class-1 hyperplane point p_1 is shown in the 2D Poincaré disk. The Riemannian optimizers move along shorter and smoother paths inside the ball, whereas the projected optimizers make larger jumps and are more often driven toward the boundary. This behavior is consistent with projection-count diagnostics: projected updates may leave the feasible ball and require correction, while Riemannian updates are geometry-aware by construction.



(a) Test F_1 vs. epoch, ANIMAL.N.01, $D = 3$.



(b) Class-1 hyperplane point trajectory, MAMMAL.N.01, $D = 2$.

Fig. 1: Convergence and stability of HMLR optimizers.

V. Discussion and Conclusion

Our results confirm the main advantage of hyperbolic classification on WORDNET subtree prediction: HMLR exploits pretrained Poincaré embeddings more effectively than Euclidean decision boundaries. In the aggregate comparison, HMLR with RAdam achieves the best mean F_1 of 0.90, clearly above the strongest Euclidean baseline, log-map LR with Adam, at 0.55. The complete tables further show that HMLR variants win 15 out of 16 subtree–dimension cells, with the only exception being GROUP.N.01 at $D = 5$.

Our contribution is not a new optimizer, but a controlled empirical comparison of how classifier geometry and optimizer geometry interact on a hierarchical prediction task. Compared with [2], our study focuses on optimization behavior: we compare projected SGD/Adam with RSGD/RAdam under the same data, embeddings, learning rate, and training budget, and analyse both F_1 curves and hyperplane-point trajectories. Riemannian optimization is more reliable on average: RAdam gives the best overall result, and RSGD outperforms projected SGD on mean F_1. However, projected methods still win some individual cells, so this should be understood as an aggregate trend rather than uniform dominance.

Embedding dimension is also important. Performance improves as dimension increases, but the $D = 2$ case shows that all four subtree tasks are already won by HMLR variants, suggesting that Poincaré hyperplanes can better exploit low-dimensional hyperbolic structure than Euclidean or tangent-space classifiers. Our study is limited to four subtrees, four dimensions, three seeds, and fixed hyperparameters; per-optimizer tuning may narrow some gaps. Within these limits, the results support using geometry-aware classifiers and Riemannian adaptive optimization for hierarchical data.

REFERENCES

[1] M. Nickel and D. Kiela, "Poincaré embeddings for learning hierarchical representations," *Advances in neural information processing systems*, vol. 30, 2017.

[2] O. Ganea, G. Bécigneul, and T. Hofmann, "Hyperbolic neural networks," *Advances in neural information processing systems*, vol. 31, 2018.

[3] R. Sarkar, "Low distortion delaunay embedding of trees in hyperbolic plane," in *International symposium on graph drawing*. Springer, 2011, pp. 355–366.

[4] F. Sala, C. De Sa, A. Gu, and C. Ré, "Representation tradeoffs for hyperbolic embeddings," in *International conference on machine learning*. PMLR, 2018, pp. 4460–4469.

[5] C. Fellbaum, *WordNet: An electronic lexical database*. MIT press, 1998.

[6] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," *arXiv preprint arXiv:1412.6980*, 2014.

[7] S. Bonnabel, "Stochastic gradient descent on riemannian manifolds," *IEEE Transactions on Automatic Control*, vol. 58, no. 9, pp. 2217–2229, 2013.

[8] G. Bécigneul and O.-E. Ganea, "Riemannian adaptive optimization methods," *arXiv preprint arXiv:1810.00760*, 2018.

[9] M. Kochurov, R. Karimov, and S. Kozlukov, "Geoopt: Riemannian optimization in pytorch," *arXiv preprint arXiv:2005.02819*, 2020.

APPENDIX

The main text reports aggregate results to save space. For transparency, this appendix reports the full seed-averaged F_1 matrices used to compute Table I. Each table contains all eight method–optimizer settings across all four embedding dimensions for one subtree. The best value in each dimension is bolded.

TABLE II: Complete seed-averaged F_1 results for ANIMAL.N.01. Each entry is mean $\pm$ standard deviation over three seeds.

Setting	$D = 2$	$D = 3$	$D = 5$	$D = 10$
Euc-LR + Adam	0.000 ± 0.000	0.956 ± 0.008	0.944 ± 0.017	0.996 ± 0.001
Euc-LR + SGD	0.000 ± 0.000	0.000 ± 0.000	0.000 ± 0.000	0.000 ± 0.000
Log-map LR + Adam	0.000 ± 0.000	0.927 ± 0.008	0.947 ± 0.006	0.992 ± 0.001
Log-map LR + SGD	0.000 ± 0.000	0.000 ± 0.000	0.344 ± 0.035	0.986 ± 0.001
H-MLR + Proj. SGD	0.687 ± 0.382	0.662 ± 0.574	0.968 ± 0.050	0.999 ± 0.001
H-MLR + Proj. Adam	0.661 ± 0.394	0.988 ± 0.008	0.998 ± 0.002	0.956 ± 0.073
H-MLR + RSGD	0.894 ± 0.008	0.992 ± 0.005	0.996 ± 0.002	0.998 ± 0.000
H-MLR + RAdam	0.901 ± 0.017	0.991 ± 0.004	0.995 ± 0.004	0.998 ± 0.002

TABLE III: Complete seed-averaged F_1 results for GROUP.N.01. Each entry is mean $\pm$ standard deviation over three seeds.

Setting	$D = 2$	$D = 3$	$D = 5$	$D = 10$
Euc-LR + Adam	0.392 ± 0.040	0.979 ± 0.004	0.952 ± 0.002	0.996 ± 0.001
Euc-LR + SGD	0.000 ± 0.000	0.829 ± 0.003	0.780 ± 0.054	0.967 ± 0.005
Log-map LR + Adam	0.575 ± 0.019	0.944 ± 0.005	0.924 ± 0.009	0.994 ± 0.000
Log-map LR + SGD	0.000 ± 0.000	0.913 ± 0.001	0.907 ± 0.003	0.987 ± 0.001
H-MLR + Proj. SGD	0.222 ± 0.385	0.982 ± 0.002	0.645 ± 0.559	0.998 ± 0.002
H-MLR + Proj. Adam	0.732 ± 0.013	0.979 ± 0.006	0.645 ± 0.559	0.994 ± 0.002
H-MLR + RSGD	0.437 ± 0.389	0.948 ± 0.047	0.948 ± 0.022	0.997 ± 0.002
H-MLR + RAdam	0.708 ± 0.076	0.974 ± 0.011	0.946 ± 0.007	0.996 ± 0.002

TABLE IV: Complete seed-averaged F_1 results for MAMMAL.N.01. Each entry is mean $\pm$ standard deviation over three seeds.

Setting	$D = 2$	$D = 3$	$D = 5$	$D = 10$
Euc-LR + Adam	0.000 ± 0.000	0.000 ± 0.000	0.000 ± 0.000	0.640 ± 0.064
Euc-LR + SGD	0.000 ± 0.000	0.000 ± 0.000	0.000 ± 0.000	0.000 ± 0.000
Log-map LR + Adam	0.000 ± 0.000	0.000 ± 0.000	0.380 ± 0.219	0.902 ± 0.014
Log-map LR + SGD	0.000 ± 0.000	0.000 ± 0.000	0.000 ± 0.000	0.000 ± 0.000
H-MLR + Proj. SGD	0.000 ± 0.000	0.327 ± 0.566	0.664 ± 0.575	0.588 ± 0.522
H-MLR + Proj. Adam	0.270 ± 0.460	0.154 ± 0.267	0.969 ± 0.041	0.518 ± 0.500
H-MLR + RSGD	0.000 ± 0.000	0.695 ± 0.085	0.750 ± 0.298	0.642 ± 0.556
H-MLR + RAdam	0.893 ± 0.099	0.957 ± 0.036	0.899 ± 0.159	0.965 ± 0.014

TABLE V: Complete seed-averaged F_1 results for WORKER.N.01. Each entry is mean $\pm$ standard deviation over three seeds.

Setting	$D = 2$	$D = 3$	$D = 5$	$D = 10$
Euc-LR + Adam	0.000 ± 0.000	0.000 ± 0.000	0.000 ± 0.000	0.849 ± 0.007
Euc-LR + SGD	0.000 ± 0.000	0.000 ± 0.000	0.000 ± 0.000	0.000 ± 0.000
Log-map LR + Adam	0.000 ± 0.000	0.000 ± 0.000	0.328 ± 0.067	0.927 ± 0.012
Log-map LR + SGD	0.000 ± 0.000	0.000 ± 0.000	0.000 ± 0.000	0.000 ± 0.000
H-MLR + Proj. SGD	0.000 ± 0.000	0.000 ± 0.000	0.321 ± 0.557	0.956 ± 0.006
H-MLR + Proj. Adam	0.000 ± 0.000	0.506 ± 0.440	0.968 ± 0.007	0.953 ± 0.007
H-MLR + RSGD	0.000 ± 0.000	0.000 ± 0.000	0.627 ± 0.543	0.953 ± 0.006
H-MLR + RAdam	0.749 ± 0.074	0.617 ± 0.534	0.777 ± 0.306	0.958 ± 0.008

TABLE VI: Mean F_1 by embedding dimension, averaged over all subtrees and settings.

Dimension	Mean F_1
$D = 2$	0.254
$D = 3$	0.510
$D = 5$	0.613
$D = 10$	0.772